

```
create database sales_datadb;
```

```
use sales_datadb;
```

```
create table sales_data (  
transaction_id int primary key,  
employee_id int,  
month varchar(50),  
sales_amount decimal(10,2));
```

```
insert into sales_data (transaction_id, employee_id, month, sales_amount) values  
(1, '101', '2024-04-01', 500.00),  
(2, '102', '2023-04-02', 1001.00),  
(3, '102', '2023-04-03', 1002.00),  
(4, '102', '2023-04-04', 1003.00),  
(5, '102', '2023-04-05', 1004.00),  
(6, '102', '2023-04-06', 1005.00),  
(7, '102', '2023-04-07', 1006.00);
```

```
select * from sales_data;
```

write a query that ranks the sales amount in decending order from the sales_sale data table ans assign a rank to each row.

HOW TO USE RANK

first - select all

second - highest sales use in >> RANK

third - over >> window fuctions use COMMON

fourth - ORDER BY WHICH AMOUNT (DESC OR ASEC) ANY OPTION

FIFTH - I HAVE TO COLUMN NAME USE >> RK

SIXTH - FINAL TABLE NAME >> WHICH MEANS --SAELS_DATA

```
select *,  
rank() over (order by sales_amount desc)  
as rk  
from sales_data;
```

OUTPUT

Result Grid   Filter Rows: Export:  Wrap Cell Content: 					
	transaction_id	employee_id	month	sales_amount	rk
▶	2	102	2023-04-02	1001.00	1
	3	102	2023-04-03	1002.00	1
	4	102	2023-04-04	1003.00	1
	5	102	2023-04-05	1004.00	1
	6	102	2023-04-06	1005.00	1
	7	102	2023-04-07	1006.00	1
	1	101	2024-04-01	500.00	1

Result 26 × 

HOW TO USE >> DENSE RANK

first - select all

second - highest sales use in >> DENSE RANK

third - over >> window fuctions use COMMON

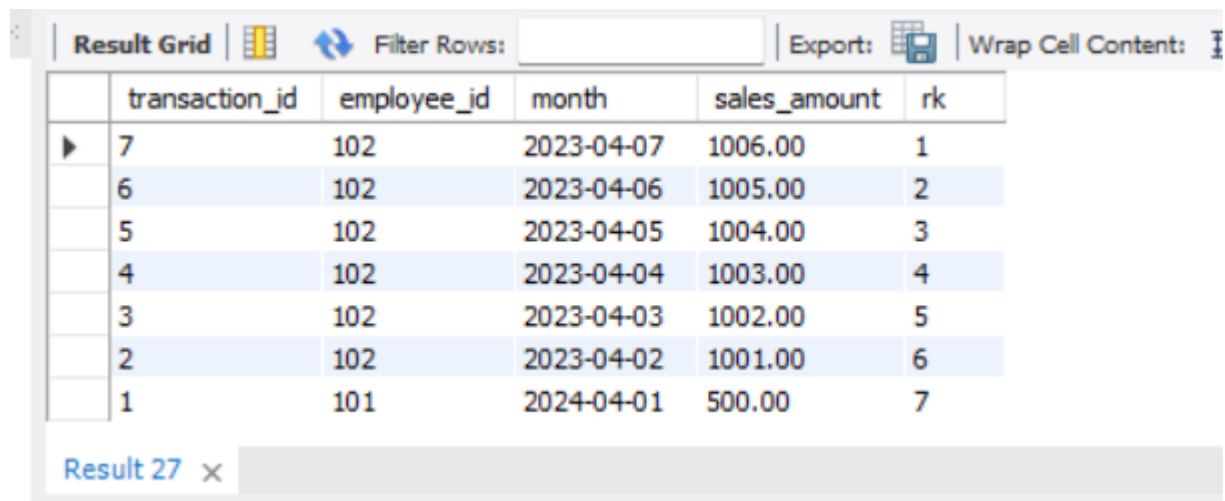
fourth - ORDER BY WHICH AMOUNT (DESC OR ASEC) ANY OPTION

FIFTH - I HAVE TO COLUMN NAME USE >> RK

SIXTH - FINAL TABLE NAME >> WHICH MEANS --SAELS_DATA

```
select *,  
DENSE_rank() over (order by sales_amount desc)  
as rk  
from sales_data;
```

OUTPUT



The screenshot shows a database query result grid with the following data:

	transaction_id	employee_id	month	sales_amount	rk
▶	7	102	2023-04-07	1006.00	1
	6	102	2023-04-06	1005.00	2
	5	102	2023-04-05	1004.00	3
	4	102	2023-04-04	1003.00	4
	3	102	2023-04-03	1002.00	5
	2	102	2023-04-02	1001.00	6
	1	101	2024-04-01	500.00	7

Result 27 x

WRITE A QUERY THAT RANK THE SALES_AMOUNT FOR EACH MONTH (PARTITION BY MONTH) IN DESCENDING ORDER FROM THE SALES_AMOUNT

HOW TO USE >> partition by month

first - select all

second - highest sales use in >> partition by month

third - over >> window fuctions use COMMON

fourth - partition by month

fifth - order by which amount (DESC OR ASEC) ANY OPTION

FIFTH - I HAVE TO COLUMN NAME USE >> RK

SIXTH - FINAL TABLE NAME >> WHICH MEANS --SAELS_DATA

select *,

rank() over (partition by month order by sales_amount desc)

as rk

from sales_data;

OUTPUT

	transaction_id	employee_id	month	sales_amount	rk
▶	2	102	2023-04-02	1001.00	1
	3	102	2023-04-03	1002.00	1
	4	102	2023-04-04	1003.00	1
	5	102	2023-04-05	1004.00	1
	6	102	2023-04-06	1005.00	1
	7	102	2023-04-07	1006.00	1
	1	101	2024-04-01	500.00	1

Result 28 ×

write a query that calculates the running total of sales_amount for each employee , order by month from the sales_data

HOW TO USE >> AGGREGATE FUNCTION >> FOR EXAMPLE- SUM()

first - select all

second - total sales amount use with >> sum()

third - over >> window fuctions use COMMON

fourth - partition by employee_id

fifth - order by month

FIFTH - I HAVE TO COLUMN NAME USE >> rk_employee_sales

SIXTH - FINAL TABLE NAME >> WHICH MEANS --SAELS_DATA

select *,

sum(sales_amount) over (partition by employee_id order by month)

as rk_employee_sales

from sales_data;

OUTPUT

transaction_id	employee_id	month	sales_amount	rk_employee_sales
1	101	2024-04-01	500.00	500.00
2	102	2023-04-02	1001.00	1001.00
3	102	2023-04-03	1002.00	2003.00
4	102	2023-04-04	1003.00	3006.00
5	102	2023-04-05	1004.00	4010.00
6	102	2023-04-06	1005.00	5015.00
7	102	2023-04-07	1006.00	6021.00

write a query that shows the sales amount for each employee from the sales_data,

PREVIOUS MONTH sales amount >> how to check

how to use >> lag()

first - select all

second - previous month sales amount use >> lag()

third - over >> window functions use COMMON

fourth - partition by employee_id

fifth - order by month

FIFTH - I HAVE TO COLUMN NAME USE >> rk_previousmonth

SIXTH - FINAL TABLE NAME >> WHICH MEANS --SAELS_data

select *,

lag(sales_amount) over (partition by employee_id order by month)

as rk_previousmonth

from sales_data;

OUTPUT

Result Grid  Filter Rows: Export:  Wrap Cell Content: 					
	transaction_id	employee_id	month	sales_amount	rk_previousmonth
▶	1	101	2024-04-01	500.00	NULL
	2	102	2023-04-02	1001.00	NULL
	3	102	2023-04-03	1002.00	1001.00
	4	102	2023-04-04	1003.00	1002.00
	5	102	2023-04-05	1004.00	1003.00
	6	102	2023-04-06	1005.00	1004.00
	7	102	2023-04-07	1006.00	1005.00

Result 30 x 

write a query that shows the NEXT MONTH sales amount for each employee from the sales_data,

NEXT MONTH sales amount >> how to check

how to use >> LEAD()

first - select all

second - NEXT month sales amount use >> LEAD()

third - over >> window functions use COMMON

fourth - partition by employee_id

fifth - order by month

FIFTH - I HAVE TO COLUMN NAME USE >> rk_NEXTmonth

SIXTH - FINAL TABLE NAME >> WHICH MEANS --SAELS_data

select *,

LEAD(sales_amount) over (partition by employee_id order by month)

as rk_NEXTMONTH

from sales_data;

OUT PUT

Result Grid Filter Rows: Export: Wrap Cell Content:					
	transaction_id	employee_id	month	sales_amount	rk_NEXTMONTH
▶	1	101	2024-04-01	500.00	NULL
	2	102	2023-04-02	1001.00	1002.00
	3	102	2023-04-03	1002.00	1003.00
	4	102	2023-04-04	1003.00	1004.00
	5	102	2023-04-05	1004.00	1005.00
	6	102	2023-04-06	1005.00	1006.00
	7	102	2023-04-07	1006.00	NULL

Result 31 x

